

Здесь будет титульник, листай ниже

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ.....	5
1.1 Описание входных данных.....	6
1.2 Описание выходных данных.....	6
2 МЕТОД РЕШЕНИЯ.....	7
3 ОПИСАНИЕ АЛГОРИТМОВ.....	9
3.1 Алгоритм конструктора класса Device.....	9
3.2 Алгоритм метода show класса Device.....	10
3.3 Алгоритм функции main.....	10
4 БЛОК-СХЕМЫ АЛГОРИТМОВ.....	12
5 КОД ПРОГРАММЫ.....	16
5.1 Файл Device.cpp.....	16
5.2 Файл Device.h.....	16
5.3 Файл main.cpp.....	17
6 ТЕСТИРОВАНИЕ.....	19
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	20

1 ПОСТАНОВКА ЗАДАЧИ

В системе центра обработки данных выполняется регистрация устройств. Каждое устройство должно удовлетворять требованиям:

- имя устройства не должно быть пустым;
- мощность устройства должна быть больше нуля;
- уровень нагрузки не должен превышать 100.

При нарушении этих условий система должна генерировать исключения.

Класс Устройство

Поля:

- строковое поле имени устройства;
- целочисленное поле мощности;
- целочисленное поле нагрузки.

Методы:

- конструктор, устанавливающий значения полей, проверяющий корректность данных, при ошибке генерирующий исключение:
 - Если имя пустое, сгенерировать исключение типа `string` со значением "Error: empty name";
 - Если мощность ≤ 0 , сгенерировать исключение типа `int` со значением "-1";
 - Если нагрузка > 100 , сгенерировать исключение типа `double` со значением "100.0"
- метод вывода информации об устройстве в формате "Device::show - > <name> <power> <load>";

Алгоритм работы программы

1. Ввести количество устройств n.
2. Для каждого устройства:

2.1. Ввести значения имени, мощности и нагрузки.

2.2. Попытаться создать объект в блоке try.

2.2.1. Если объект создан успешно, вывести его данные.

2.2.2. Если возникло исключение, обработать его соответствующим блоком catch и вывести сообщение об ошибке.

Формат обработки исключений

- для `string`: `Exception(string): <текст>`
- для `int`: `Exception(int): <значение>`
- для `double`: `Exception(double): <значение>`

Для корректного чтения имени допустимо использовать `getline()`.

1.1 Описание входных данных

Пример:

```
3
Router 50 30
0 20
Server -10 40
```

1.2 Описание выходных данных

Пример:

```
Device::show -> Router 50 30
Exception(string): Error: empty name
Exception(int): -1
```

2 МЕТОД РЕШЕНИЯ

Для решения задачи используется:

- объект `cin` класса `istream` предназначен для ввода с клавиатуры;
- объект `d` класса `Device` предназначен для создания объекта класса;
- объект `iss` класса `istringstream` предназначен для разбора текста на части;
- функция `printf` для форматированного вывода данных;
- функция `getline` для считывания строки текста из входного потока;
- `try...catch` - конструкция для обработки исключений;
- `if...else` - условный оператор;
- `throw` - оператор создания исключений;
- `for` - оператор цикла со счетчиком.

Класс `Device`:

- свойства/поля:
 - о поле название устройства:
 - наименование — `name`;
 - тип — `string`;
 - модификатор доступа — `private`;
 - о поле мощность:
 - наименование — `N`;
 - тип — `int`;
 - модификатор доступа — `private`;
 - о поле нагрузка:
 - наименование — `P`;
 - тип — `int`;
 - модификатор доступа — `private`;

- функционал:
 - метод `Device` — конструктор с параметрами;
 - метод `show` — метод вывода закрытых свойств объекта.

3 ОПИСАНИЕ АЛГОРИТМОВ

Согласно этапам разработки, после определения необходимого инструментария в разделе «Метод», составляются подробные описания алгоритмов для методов классов и функций.

3.1 Алгоритм конструктора класса Device

Функционал: конструктор с параметрами.

Параметры: string name - название устройства, int N - мощность, int P - нагрузка.

Алгоритм конструктора представлен в таблице 1.

Таблица 1 – Алгоритм конструктора класса Device

№	Предикат	Действия	№ перехода
1	name.empty()	создаем исключение "ErrorL empty name"	∅
			2
2	N <= 0	создаем исключение -1	∅
			3
3	P > 100	создаем исключение 100	∅
			4
4		присвоение закрытому свойству name значения параметра name	5
5		присвоение закрытому свойству N значения параметра N	6
6		присвоение закрытому свойству P значения параметра P	∅

3.2 Алгоритм метода show класса Device

Функционал: метод вывода закрытых свойств объекта.

Параметры: нет.

Возвращаемое значение: void - ничего не возвращает.

Алгоритм метода представлен в таблице 2.

Таблица 2 – Алгоритм метода show класса Device

№	Предикат	Действия	№ перехода
1		вывод на экран "Device::show -> <name> <power> <load>"	Ø

3.3 Алгоритм функции main

Функционал: основной алгоритм программы.

Параметры: нет.

Возвращаемое значение: int - индикатор корректности завершения программы.

Алгоритм функции представлен в таблице 3.

Таблица 3 – Алгоритм функции main

№	Предикат	Действия	№ перехода
1		инициализация переменной n типа int	2
2		ввод с клавиатуры n	3
3		вызов метода ignore() объекта cin	4
4	i < n	инициализация переменных line, name типа string	5
			Ø
5		инициализация переменных N, P типа int	6
6		вызов функции getline() с параметрами std::cin, line	7
7		инициализация объекта iss класса istream с параметром line	8

№	Предикат	Действия	№ перехода
8	!(iss >> name >> N >> P)	присвоение переменной p значения ""	9
			9
9	try	инициализация объекта d класса Device с параметрами name, N, P	10
			10
10	catch (std::string e)	вывод на экран "Exception(string): <текст>"	4
			11
11	catch (int e)	вывод на экран "Exception(int): <значение>"	4
			12
12	catch (double e)	вывод на экран "Exception(double): <значение>"	4
			13
13		вызов метода show() от объекта d	4

4 БЛОК-СХЕМЫ АЛГОРИТМОВ

Представим описание алгоритмов в графическом виде на рисунках 1-4.

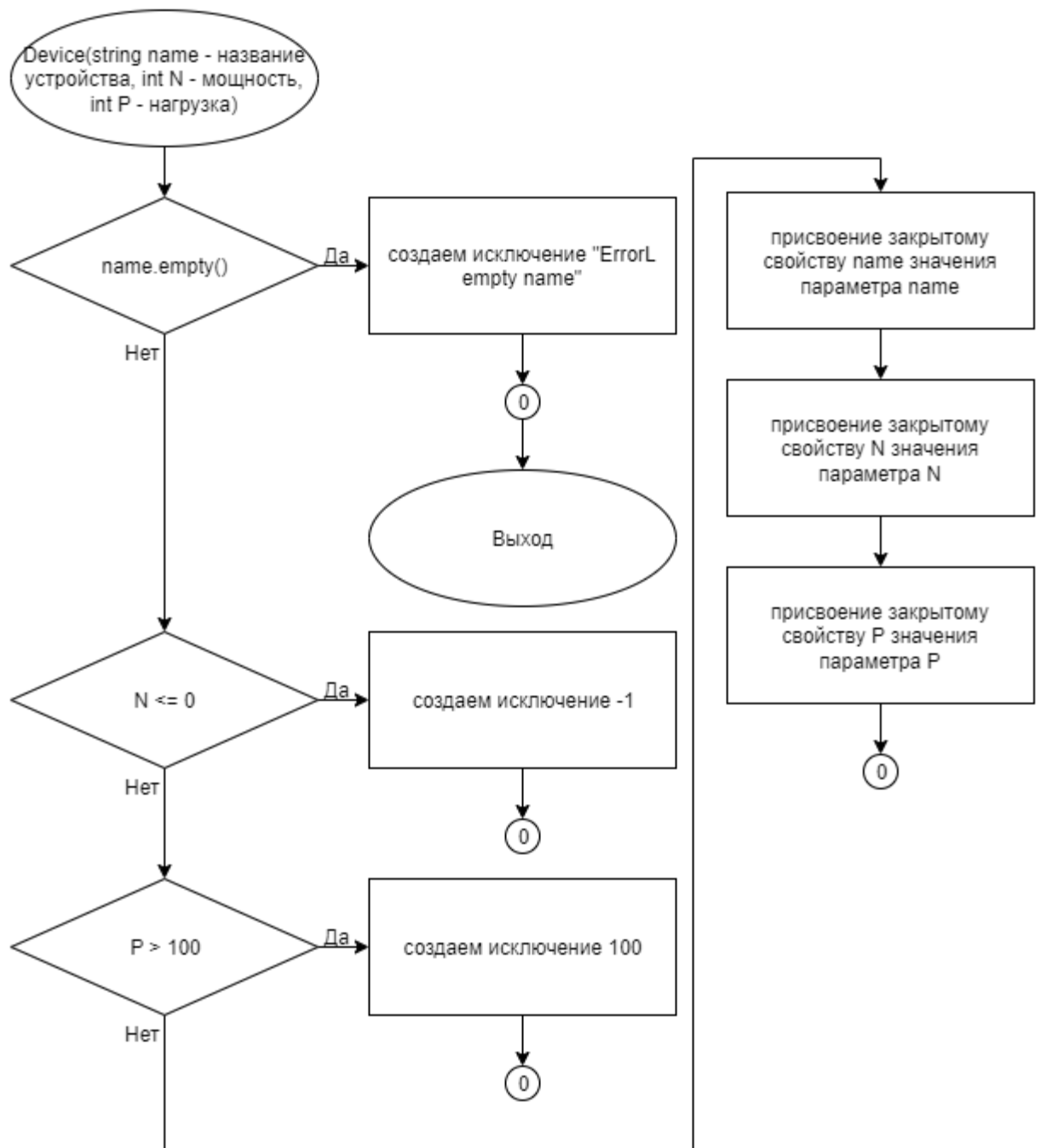


Рисунок 1 – Блок-схема алгоритма

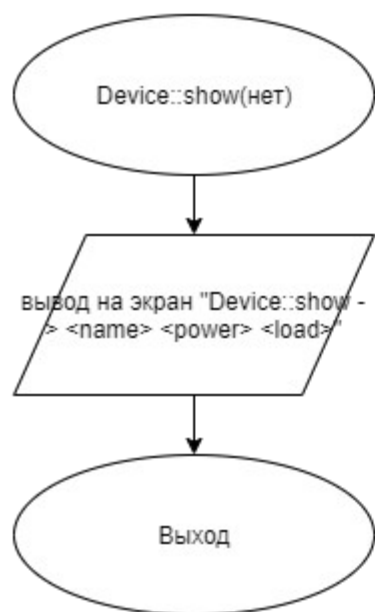


Рисунок 2 – Блок-схема алгоритма

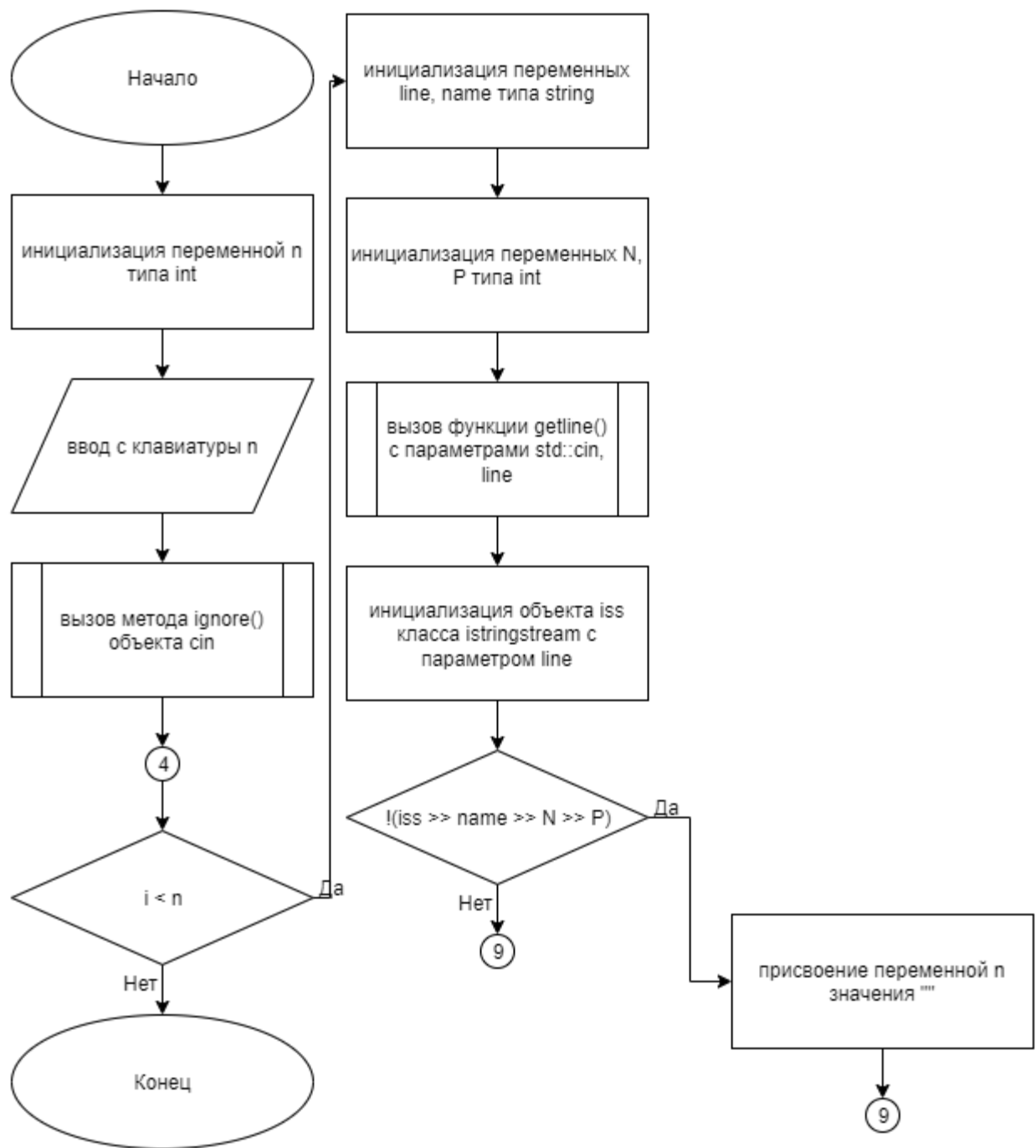


Рисунок 3 – Блок-схема алгоритма

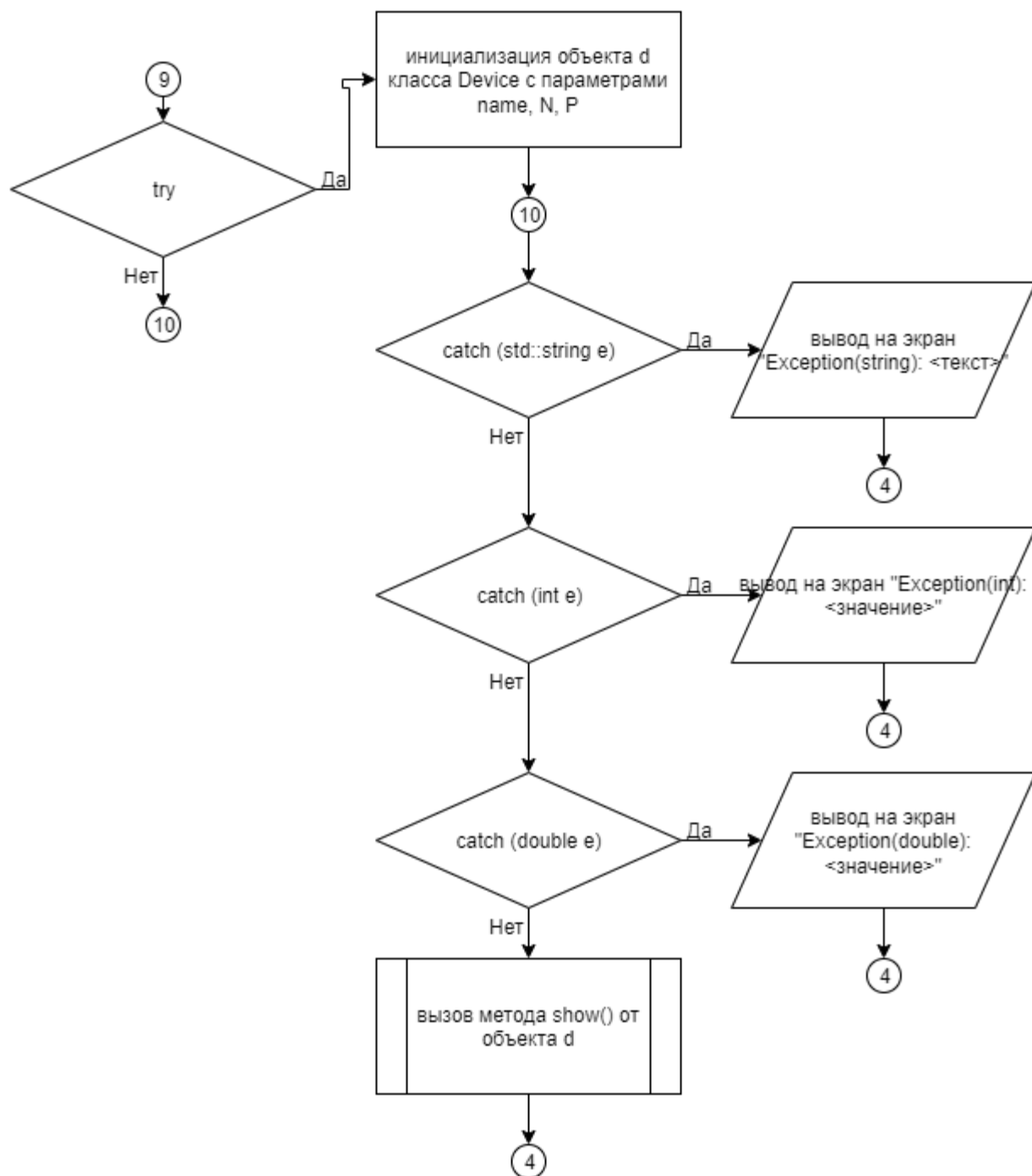


Рисунок 4 – Блок-схема алгоритма

5 КОД ПРОГРАММЫ

Программная реализация алгоритмов для решения задачи представлена ниже.

5.1 Файл Device.cpp

Листинг 1 – Device.cpp

```
#include "Device.h"
#include <string>

Device::Device(std::string name, int N, int P)
{
    if (name.empty())
        throw std::string("Error: empty name");

    if (N <= 0)
        throw -1;

    if (P > 100)
        throw 100.;

    this->name = name;
    this->N = N;
    this->P = P;
}

void Device::show()
{
    printf("Device::show -> %s %d %d\n", name.c_str(), N, P);
}
```

5.2 Файл Device.h

Листинг 2 – Device.h

```
#ifndef __DEVICE__H
#define __DEVICE__H

#include <string>

class Device
```

```

{
private:
    std::string name;
    int N, P;

public:
    Device(std::string name, int N, int P);
    void show();
};

#endif

```

5.3 Файл main.cpp

Листинг 3 – main.cpp

```

#include <stdlib.h>
#include <stdio.h>
#include <iostream>
#include <sstream>
#include <string>
#include "Device.h"

int main()
{
    int n;
    std::cin >> n;
    std::cin.ignore();

    for (int i = 0; i < n; i++)
    {
        std::string line;
        std::getline(std::cin, line);

        std::istringstream iss(line);

        std::string name;
        int N, P;

        if (!(iss >> name >> N >> P))
            name = "";

        try
        {
            Device d(name, N, P);
            d.show();
        }
        catch (std::string e)
        {
            printf("Exception(string): %s\n", e.c_str());
        }
    }
}

```

```
    }  
    catch (int e)  
    {  
        printf("Exception(int): %d\n", e);  
    }  
    catch (double e)  
    {  
        printf("Exception(double): %.f\n", e);  
    }  
}  
return(0);  
}
```


6 ТЕСТИРОВАНИЕ

Результат тестирования программы представлен в таблице 4.

Таблица 4 – Результат тестирования программы

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
3 Router 50 30 0 20 Server -10 40	Device::show -> Router 50 30 Exception(string): Error: empty name Exception(int): -1	Device::show -> Router 50 30 Exception(string): Error: empty name Exception(int): -1
3 Router 50 111 Atmega 8 20 Server -10 112	Exception(double): 100 Device::show -> Atmega 8 20 Exception(int): -1	Exception(double): 100 Device::show -> Atmega 8 20 Exception(int): -1

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19 Единая система программной документации.
2. Методическое пособие студента для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [Электронный ресурс] – URL: https://mirea.aco-avvora.ru/student/files/methodichescoc_posobie_dlya_laboratornyh_robot_3.pdf (дата обращения 05.05.2021).
3. Приложение к методическому пособию студента по выполнению заданий в рамках курса «Объектно-ориентированное программирование» [Электронный ресурс]. URL: https://mirea.aco-avvora.ru/student/files/Prilozheniye_k_methodichke.pdf (дата обращения 05.05.2021).
4. Шилдт Г. С++: базовый курс. 3-е изд. Пер. с англ.. — М.: Вильямс, 2019. — 624 с.
5. Видео лекции по курсу «Объектно-ориентированное программирование» [Электронный ресурс]. АСО «Аврора».
6. Антик М.И. Дискретная математика [Электронный ресурс]: Учебное пособие /Антик М.И., Казанцева Л.В. — М.: МИРЭА — Российский технологический университет, 2018 — 1 электрон. опт. диск (CD-ROM).